

Rethinking Cryptography in the Linux Kernel

Preparing for the Post-Quantum (PQC) Era

Yunseong Kim <ysk@kzalloc.com>



- Quantum computers can break RSA, ECC, and other algorithms the kernel relies on.
- If we don't adapt, core security features like secure boot and kernel module signing become vulnerable.
- **Quantum is coming**, We need post-quantum crypto—now, not later.
- **Kernel and other legacy-crypto-based projects must evolve.**

- Shor's Algorithm — Total Collapse of Public-Key Crypto
- Efficiently factors large integers and solves discrete logarithms.
- **RSA, ECDSA, ECC (Public-Key)** → all can be completely broken.
- How many qubits are needed?
 - Breaking RSA-2048 requires about 4,000 logical qubits.
 - With error correction, that means millions of physical qubits.
- As qubit count increases:
 - RSA-1024 → breakable in minutes
 - RSA-2048 → becoming practically vulnerable
- Once quantum hardware scales, public-key crypto collapses
→ we need post-quantum algorithms now.

- Grover's Algorithm — Weakens Symmetric and Hash Algorithms
- In classical computing, searching an unsorted list takes $O(n)$ time.
- Grover reduces this to $O(\sqrt{n})$ on a quantum computer.
- In cryptography:
 - Brute-forcing an n -bit symmetric key takes $O(2^n)$ time classically.
 - With Grover's Algorithm, this is reduced to $O(2^{n/2})$ (i.e., $\sqrt{2^n}$).
 - So, the phrase " $2^n \rightarrow \sqrt{2^n}$ " means Grover halves the number of bits of security in symmetric encryption and hashing.
- AES (symmetric ciphers), SHA-2, SHA-3 (hash functions) affected.
- Impact of quantum search:
 - AES-128 $\rightarrow$ reduced to $\sim 2^{64}$ effort
 - SHA-256 $\rightarrow$ reduced to $\sim 2^{128} \rightarrow \sim 2^{64}$ for collision resistance
- As qubits increase:
 - AES-128 needs ~ 128 logical qubits
 - SHA-256 needs ~ 256 logical qubits
- More qubits = exponentially faster brute-force attacks
- Not fully broken, but security is halved $\rightarrow$ use longer keys and hashes.

- Block Ciphers (e.g., **AES-128**)
- Threat: **Grover's algorithm** can halve effective key strength.
- Action: Use longer keys (e.g., **AES-256**).

- Hash Functions (e.g., **SHA-2**)
- Threat: **Grover's algorithm** reduces security level from n to $\sqrt{n}$.
- Action: Use stronger hashes (e.g., **SHA-512** or **SHA-3**).

- Public-Key Crypto (e.g., **RSA**, **ECC**)
- Threat: **Shor's algorithm** can completely break RSA and ECC.
- Action: Replace with post-quantum algorithms (e.g., **ML-KEM**, **ML-DSA**).

1976 – Diffie-Hellman key exchange

1977 – RSA cryptosystem

1978 – Code-based cryptography

1979 – Hash-based cryptography

1980 – Realization that a quantum computer can simulate things a classical computer cannot

1984 – Quantum Key Distribution (QKD)

1985 – Elliptic Curve Cryptography (ECC)

1986 – Grover's quantum algorithm: inverts any function using only N evaluations of the function

1994 – Shor's quantum algorithm: integer factorization in polynomial time instead of sub-exponential

1996 – Multivariate cryptography

1998 – Lattice-based cryptography

1998 – First working quantum computer with 2 physical qubits

2001 – First quantum key distribution network

2011 – Elliptic Curve Isogeny Cryptography

2015 – NSA announced that it is planning a transition to quantum-resistant cryptography

2017 – **NIST announces Post-Quantum Cryptography (PQC) standardization program**

2018 – Standardization of stateful hash-based signatures (XMSS, LMS) by IETF and NIST

2022 – **US government (NSA) announced the quantum-resistant CNSA 2.0 suite required for National Security Systems**

2024 – Liaison statement: Response to the Liason Statement on Quantum Safe Cryptographic Protocol Inventory to the IETF

Symmetric algorithms with at least 128-bit keys (such as AES) can continue to be used.

2025 – “Guidelines on Cryptographic Algorithms Usage and Key Management” by the European Payments Council

SLH-DSA (e.g., SPHINCS+), **Classic McEliece**, and FrodoKEM are recommended. A hybrid approach (classical + post-quantum) is required.

2024 – NIST publishes final standards of **ML-KEM**, **ML-DSA**, and **SLH-DSA**

2024 – NIST states that non-quantum-resistant asymmetric crypto will be **deprecated 2030 and disallowed 2035.**

2025 – NIST officially announced the selection of **HQC (Hamming Quasi-Cyclic)** as the next code-based **KEM (Key Encapsulation Mechanism)** standard.

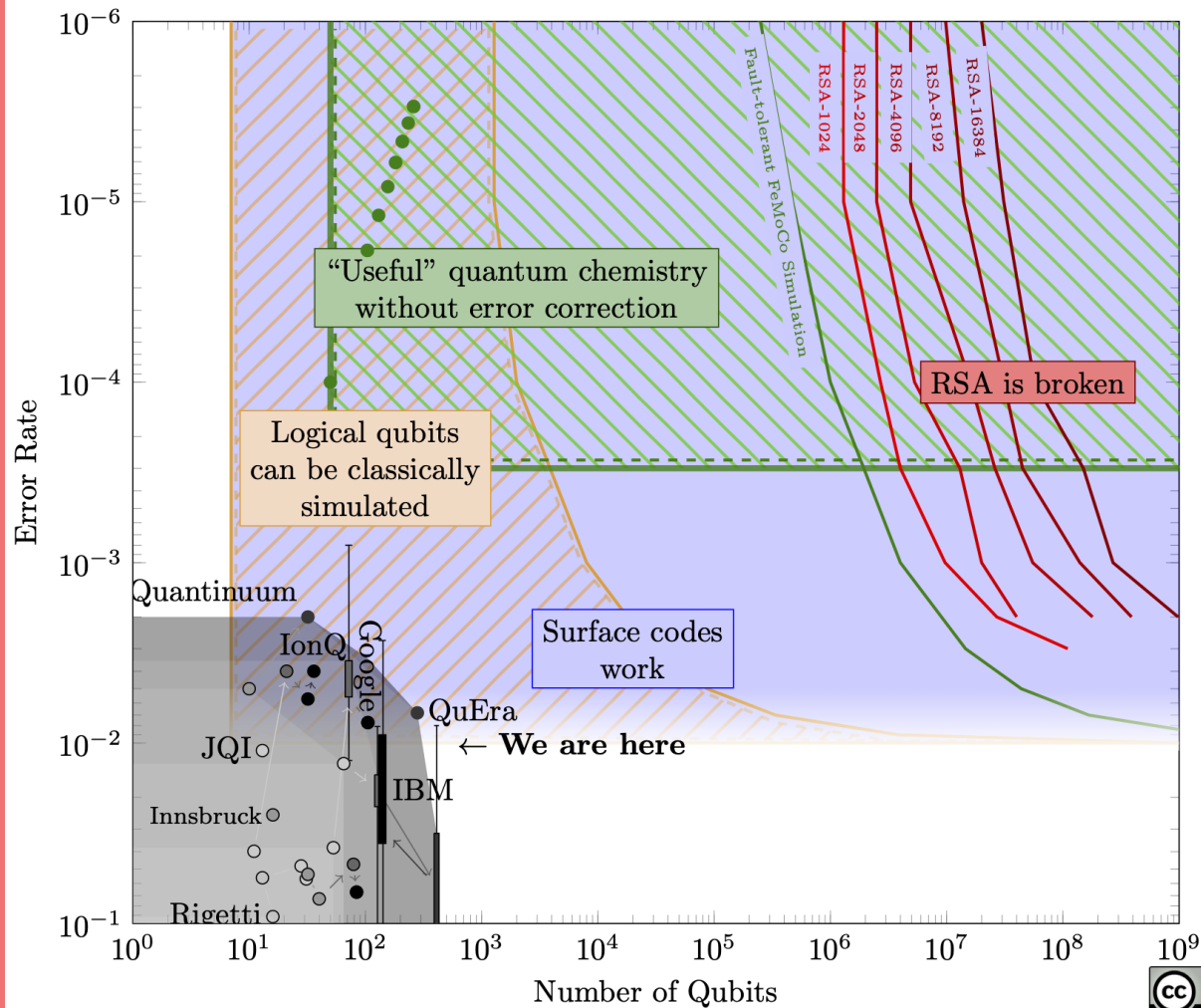
BIKE and Classic McEliece were not selected for standardization in this round.

NIST is actively developing detailed migration and transition guidelines, with draft recommendations released in late 2024 and undergoing public comment through 2025.

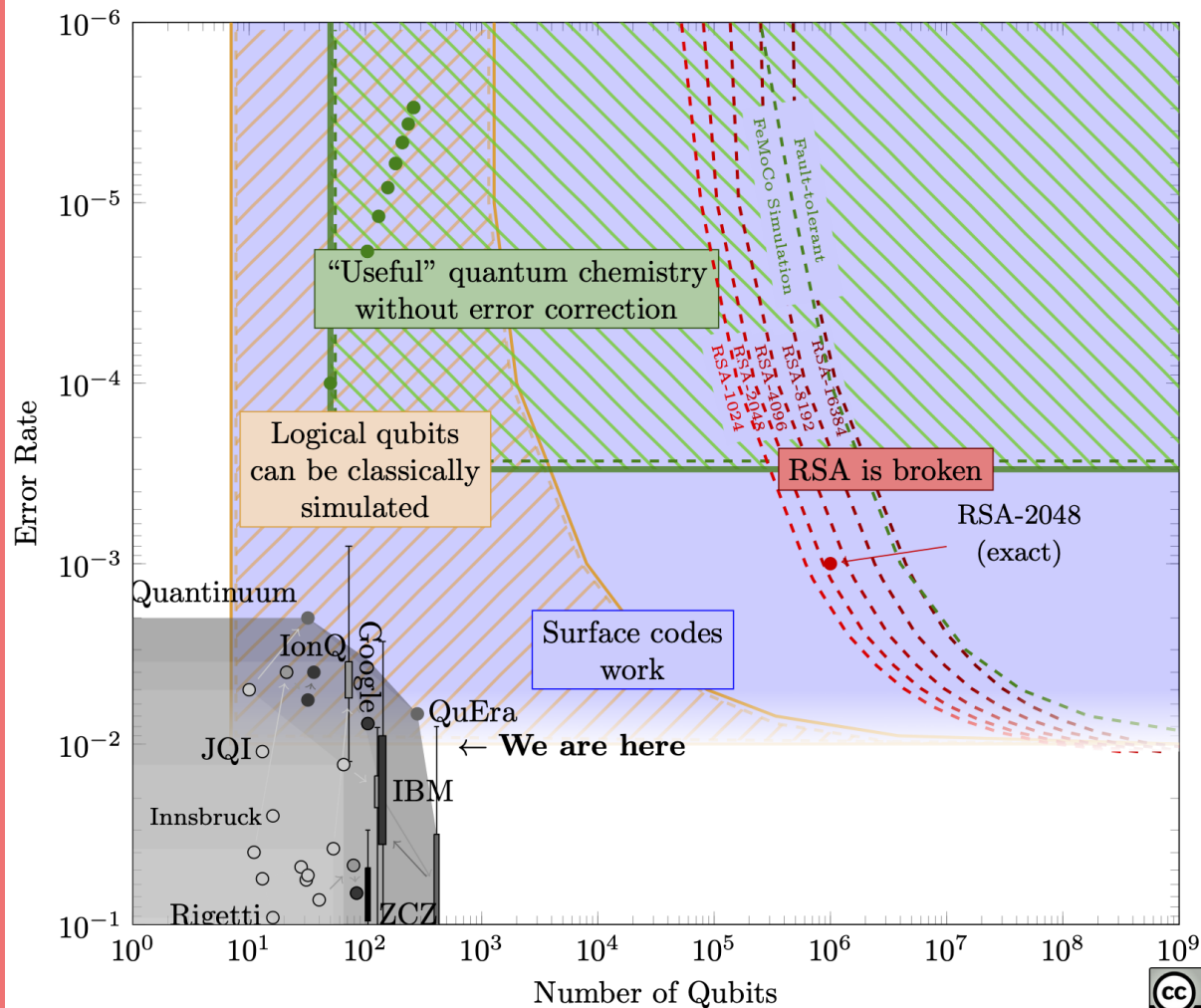
These outline phased deprecation schedules and best practices for post-quantum readiness.

- **Shor's quantum algorithm** on a large and robust quantum computer would break the asymmetric crypto algorithms (RSA, ECC) we use today.
 - Such a quantum computer is called a cryptographically relevant quantum computer (CRQC).
- A CRQC requires tens of millions of robust physical qubits and trillions of quantum gates.
 - The largest quantum computer currently has around 1000 unstable physical qubits.
- It is unclear when or if CRQCs will be built. The emergence of a CRQC in the coming 10 years would be very unexpected.
- If the number of qubits doubles every two years, it takes 30 years.
 - Some signs that investment is slowing down.
- IBM's roadmap for 2033+ is now 2000 qubits.
- Experimental breakthrough in error correction.
 - https://sam-jaques.appspot.com/quantum_landscape_2025
 - <https://youtu.be/eB4po9Br1YY?si=B1JFKwi7QXpnO4en>

Landscape of Quantum Computing in 2024



Landscape of Quantum Computing in 2025



- “bits of security” does not correlate with practical security when attacks cannot be effectively parallelized.
- A CRQC that can break RSA in a few hours would not pose any practical threat at all to symmetrical algorithms such as AES-128 and SHA-256. Even with millions of CRQCs it would take millions of years.
- AES-128, SNOW 3G, Keccak, and SHA-256 in the Linux kernel will remain secure for the foreseeable future.
- **Harvest Now, Decrypt Later threat**
- **Compatibility issues:** Some legacy software or middleboxes may not recognize new PQC keys and could produce errors

- AWS
 - <https://github.com/aws/aws-lc>
 - <https://github.com/aws/s2n-tls>
- CloudFlare
 - <https://github.com/cloudflare/boringssl-pq>
- A PQC library jointly developed by Google, MS, Cloudflare, and others
 - <https://github.com/open-quantum-safe/liboqs>
 - <https://cloud.google.com/blog/products/identity-security/why-google-now-uses-post-quantum-cryptography-for-internal-comms?hl=en>
- The EU and the US Department of Defense have also outlined roadmaps recommending PQC adoption, and with NIST finalizing the standards in 2024, the migration has now officially begun.



- OpenSSH 10.0/10.0p2 (2025-04-09) New features

* ssh(1): the hybrid **post-quantum** algorithm **mlkem768x25519-sha256** is now used by **default** for **key agreement**. This algorithm is considered to be safe against attack by quantum computers, is guaranteed to be no less strong than the popular **curve25519-sha256** algorithm, has been standardised by NIST and is considerably faster than the previous default.

- <https://www.openssh.com/releases/notes.html>

- NIST has defined five security levels for quantum-resistance:

Level	Security Description
1	At least as hard to break as AES128 (exhaustive key search)
2	At least as hard to break as SHA256 (collision search)
3	At least as hard to break as AES192 (exhaustive key search)
4	At least as hard to break as SHA384 (collision search)
5	At least as hard to break as AES256 (exhaustive key search)

- The idea that symmetric cryptography will be practically affected by CRQCs is now seen as a misconception.
- The “bits of security” concept does not work with algorithms that are not parallelizable and NIST is therefore transitioning to quantumresistant security levels based on symmetric algorithms where level 1 is equivalent with AES-128, level 2 is SHA-256, etc.
- UK goverment: AES with ≥ 128 -bit keys is safe even in the post-quantum world.
- Grover’s best-case attack: $O(2^{64})$ for AES-128
- Needs: 10^9 CRQCs $\times$ \$1B each CRQC
- Runtime: ~ 1 million years (uninterrupted)
- Symmetric crypto is safe – focus on replacing asymmetric algorithms first.
- Classical brute-force: $O(2^n)$
- Grover’s quantum search: $O(2^{n/2})$
- For AES-256: Even Grover requires $O(2^{128})$ operations $\rightarrow$ still infeasible
 $\Rightarrow$ No practical quantum threat to symmetric crypto
- A very good summary of the impact of Cryptographically Relevant Quantum Computers (CRQCs) on symmetric cryptography was recently given in a statement by the Internet Engineering Task Force (IETF).

- The IETF statement refers to UK NCSC whitepaper that says symmetric algorithms with at least 128-bit keys (such as AES) can continue to be used.
- Most other 128-bit algorithms such as SNOW 3G, ZUC, TUAKE, KMAC128, Ascon, etc. are likely to have similar quantum overhead for Grover's algorithm which is known to be optimal.
- Even if an algorithm has slightly lower quantum overhead than AES-128, SA3 believes the algorithm would still fulfill the requirement (comparable to key search on AES-128) for quantum resistance category 1
- Sam Jaques keynote at CHES 2024: "Qubits would cover the moon"
- NIST 2024: "All NIST-approved symmetric primitives that provide at least 128 bits of classical security are believed to meet the requirements of at least Category 1 security"
- Making something quantum-resistant means supporting quantum-resistant public-key algorithms instead of RSA and ECC everywhere.
- Migration involves inventory, crypto agility, prioritization, development, deployment, and to disable standalone RSA/ECC.

- Post-Quantum Cryptography (PQC) is public key cryptography that is secure against CRQCs.
 - Can run in software on classical computers just like RSA and ECC.
 - All governments trusts code-based (1978), hash-based (1979), and lattice-based (1998) cryptography.
- Standardization and discussions are about which optimisations that should be made.

- IETF (and later NIST) standardized stateful hash-based signatures (XMSS and LMS) in 2018.
 - NIST **forbidding private key export is a major issue.**
 - NIST working on update to SP 800-208.
- NIST standardized **ML-KEM** (lattice), **ML-DSA** (lattice), and **SLH-DSA** (stateless hash) in 2024.
 - FN-DSA (lattice) is expected to be published in 2025.
 - NIST officially selected **HQC (Hamming Quasi-Cyclic)** for standardization,
 - Round 2 of the additional quantum-resistant signatures project continues in 2025.
- ISO is working on standardization of Classic McEliece, **ML-KEM**, and FrodoKEM (unstructured lattice). Paywalled.

- US national security systems will use XMSS, LMS, ML-KEM, and ML-DSA without hybrid. NIST allows hybrid.
- European governments also recommends SLH-DSA, Classic McEliece and FrodoKEM. Requires hybrid.

- PQC migration is similar to the migration from 3DES to AES and SHA-1 to SHA-2. Difference is that there is no clear end year.
- What are the vulnerable algorithms RSA and ECC primarily used for?
 - Key exchange/encapsulation/encryption
 - Binding signatures for non-repudiation
 - Signatures/root-of-trust for soft-/firmware upgrades
 - Short-term signatures for authentication
 - Signatures/ root-of-trust in PKI

- When to stop using RSA and ECC depends on the protection lifetime and the availability of CRQCs. Protection lifetimes vary from days to decades.
- If CRQCs are ever built, early CRQRs will likely be very expensive and only used in targeted attacks to recover keys and ciphertexts that are of particular interest.
- Software is easier to update than hardware.

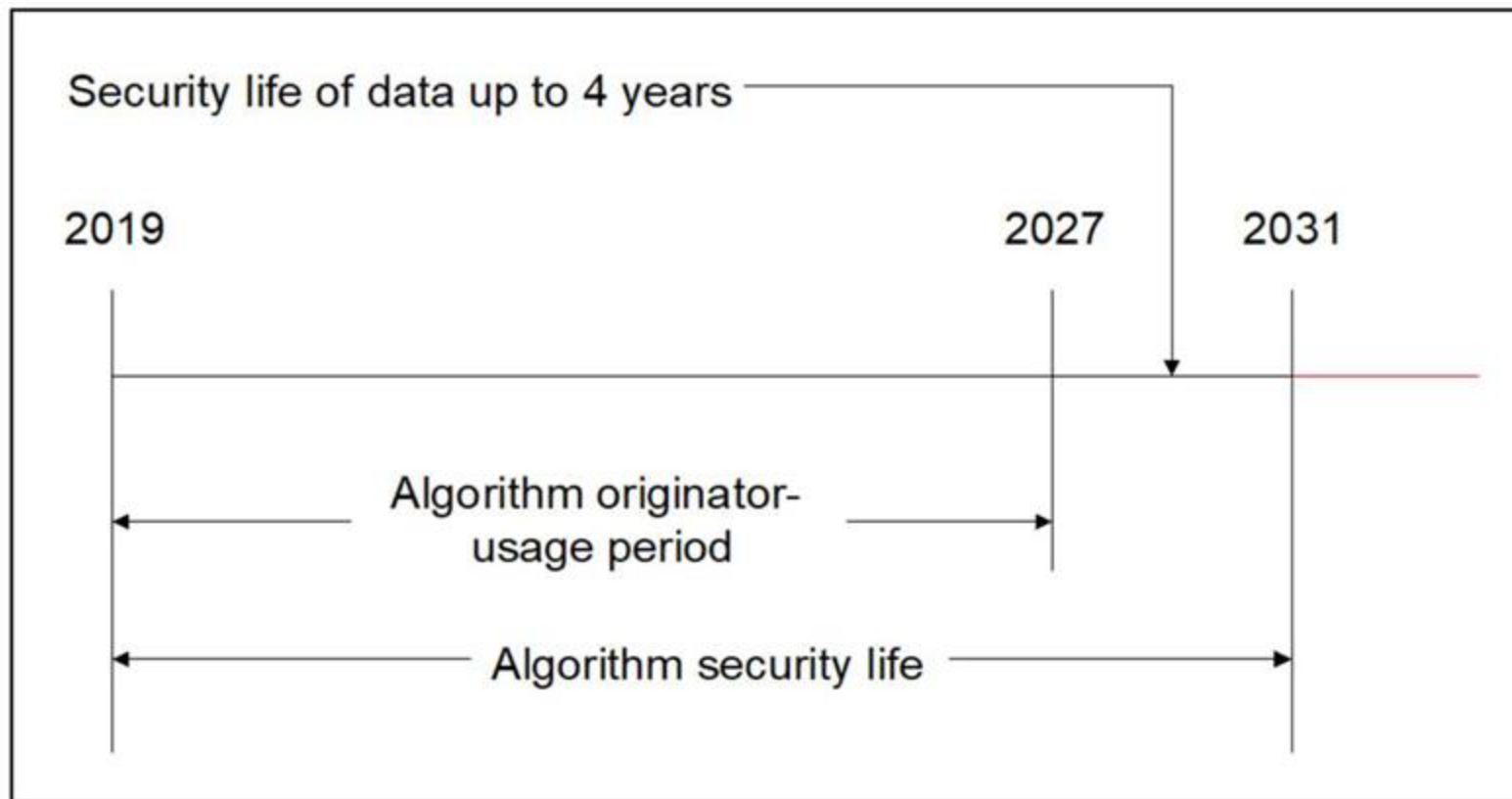


Figure 2: Algorithm originator-usage period example

- In addition to the protocols in the connectivity layer the application layer also use DTLS-SRTP and MIKEY-SAKKE.
- No PQC replacement for identity-based, attribute-based, and privacy-enhancing cryptography (pairing-based crypto).
- PQC sizes will be problematic/impossible for constrained IoT.

- In addition to the protocols in the connectivity layer the infrastructure layer also uses SSH and MACsec keyed with EAP-TLS
- Signatures for secure boot, vendor certificates, remote attestation, firmware updates, software signing, etc, ...
- Work on quantum-resistant cryptography in infrastructure layer hardware has already started many years ago.

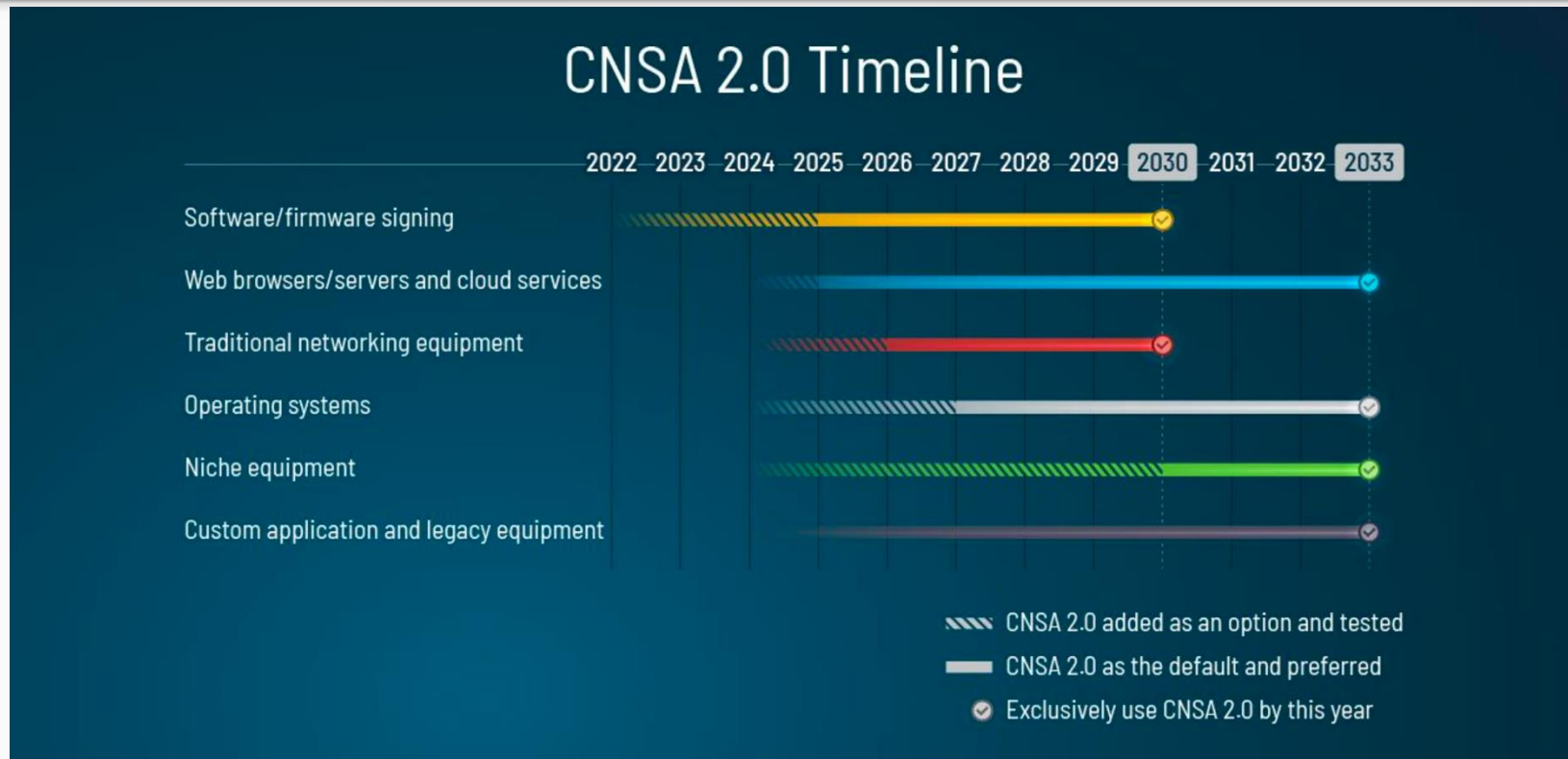


Figure 1: Transition timeline

https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF

- US government has published timelines for the migration of national security systems to PQC.
- NIST states that non-quantum-resistant asymmetric crypto will be deprecated 2030.
- European countries (e.g., ANSSI) also recommend migration to PQC by/around 2030.
- US has selected **ML-KEM** and **ML-DSA** with **security level V** as the general **future public-key algorithms**.
 - Also recommended by UK, Australia, Canada, New Zealand, Germany, France, Netherlands.
- The CNSA suite is used to protect classified information that often must stay confidential for 50+ years.

- Key Encapsulation Mechanisms (KEMs)

- NIST (USA): Standardizing ML-KEM; plans to standardize one or more code-based KEMs.
- CNSA 2.0 (USA): Requires ML-KEM-1024 (Level 5) for National Security Systems (NSS).
- NCSC (UK): Recommends ML-KEM-768 (Level 3).
- BSI (Germany): Recommends ML-KEM-768 and ML-KEM-1024, FrodoKEM-976, FrodoKEM-1344, McEliece460896, McEliece6688128, McEliece8192128
- ANSSI (France): Recommends ML-KEM-768, ML-KEM-1024, and FrodoKEM as a conservative option.
- NLNCSA (Netherlands): Recommends ML-KEM-768 and considers Classic McEliece and FrodoKEM
- South Korea and China: Developing their own PQC algorithms, differing from those of NIST.

- Signature Algorithms

- NIST (USA): Standardizing ML-DSA, FN-DSA, SLH-DSA, and potentially one more after Round 4.
- CNSA 2.0 (USA): Recommends ML-DSA-87 (Level 5) as a general signature and LMS/XMSS (single-tree) for firmware signing.
- NCSC (UK): Recommends ML-DSA-65, SLH-DSA, and LMS/XMSS for long-term signatures.
- BSI (Germany): Recommends SLH-DSA or ML-DSA (Levels 3 and 5), and LMS/XMSS in multi-tree variants for long-term signatures.
- ANSSI (France): Recommends ML-DSA, FN-DSA (Levels 3 and 5), SLH-DSA, and LMS/XMSS.
- NLNCSA (Netherlands): Recommends all NIST signatures and LMS/XMSS.

- ML-KEM is a lattice-based key encapsulation mechanism and can be used as a drop-in replacement for key exchange in TLS and IPsec.
- ML-KEM-1024 will be used by US government for protection of TOP SECRET information.
- Large sizes are problematic in some use cases.
- Classic McEliece often has the best performance for static authentication keys (S/MIME, **WireGuard**, EDHOC, ...) and is considered very conservative.

Algorithm	Security Level	TLS key share size (in bytes)		Operation per s (higher is better)	
		Client	Server	Client	Server
X25519	✗	32	32	19,000	19,000
ML-KEM-512	I	800	768	45,000	70,000
ML-KEM-768	III	1,184	1,088	29,000	45,000
ML-KEM-1024	V	1,568	1,568	20,000	30,000
mceliece348864	I	261,120	96	62,000	14,000

<https://blog.cloudflare.com/pq-2024/#ml-kem-versus-x25519>

While the code-based Classic McEliece uses binary Goppa codes, the code-based BIKE and HQC use QC-MDPC codes. BIKE and HQC are good alternatives to ML-KEM for ephemeral key exchange.

- Table 1. KEM Public key and ciphertext sizes in bytes. Total size is public key size plus ciphertext size which is a relevant measure when KEMs are used for ephemeral key exchange in protocols like TLS 1.3 and IKEv2.
- Table 2. KEM performance in cycles (50%) on 2023 AMD Ryzen 7 7700. Performance number from eBACS: ECRYPT Benchmarking of Cryptographic Systems by Bernstein and Lange (editors)
 - Total is cycles for key gen + encapsulation + decapsulation.
 - <https://bench.cr.yp.to/results-kem/amd6hertz.html>

Name	Category	Public key	Ciphertext	Total
ML-KEM-512	1	800	768	1568
BIKE-L1	1	1541	1573	3114
ML-KEM-512+BIKE-L1	1	2341	2341	4682
HQC-128	1	2249	4481	6730
ML-KEM-512+HQC-128	1	3049	5249	8298
FrodoKEM-640	1	9616	9720	19336

Name	Category	Key Gen	Encapsulation	Decapsulation	Total
ML-KEM-512 (kyber512)	1	15420	24443	18693	58556
HQC-128 (hqc128round4)	1	61311	170433	283249	514993
ML-KEM-512+HQC-128	1	76731	194876	301942	573549
BIKE-L1 (bikel1)	1	459202	83286	1069392	1611880
ML-KEM-512+BIKE-L1	1	474622	107729	1088085	1670436
FrodoKEM-640 (frodokem640shake)	1	2084314	2265633	2222733	6572680

- ML-DSA-87 (security level V) will be used by US government in national security systems.
- Large sizes and slow performance are problematic in some use cases.
- NIST has new competition for multivariate, isogeny, and lattice-based signatures.

	Security level	Sizes (bytes)		CPU time (lower is better)	
		Public key	Signature	Signing	Verification
Ed25519	✗	32	64	1 (baseline)	1 (baseline)
RSA-2048	✗	384	384	70	0.3
ML-DSA-44	II	1,312	2,420	4.8	0.5
FN-DSA-512	I	897	666	8*	0.5
SLH-DSA-128s	I	32	7,856	8,000	2.8
SLH-DSA-128f	I	32	17,088	550	7
MAYO ₁	I	1,168	321	4.7	0.3
MAYO ₂	I	5,488	180	5	0.2
UOV Is	I	66,576	96	2.5	2
SQISign I	I	64	177	60,000	500
HAWK-512	I	1,024	555	2	1

- IETF is planning to add ML-KEM and ML-DSA to all major protocols. IETF will recommend hybrids.
 - X25519MLKEM768 is the new de facto standard key exchange method in TLS 1.3
 - IKEv2 will always use hybrid key exchange for technical reasons.
 - IETF is discussing if two certificate chains or one certificate chain with composite keys and signatures should be used.
- DNSSEC and the Web will likely take a wait-and-see approach. May use MAYO (multivariate) or HAWK (lattice) security level I. Sometimes called PQC generation 2 and is likely 5 years from standardization.
- European governments might standardize the use of more conservative algorithms (FrodoKEM, Classic McEliece) with much slower performance in IETF protocols.
- Hybrid modes is a large discussion area. Hybrid modes means that PQC is used together with RSA or ECC.
 - US government says that hybrid is not recommended as it increases complexity.
 - European governments say that hybrid is a must due to possibility of theoretical or implementation attacks.
- Sizes are a major issue

Table 1: Public-Key Algorithm Use Cases in the Linux Kernel

Algorithm	Primary Kernel Subsystem/Use Case	Purpose of Use	Key Kernel Config/Module	Status/Notes
RSA	Kernel Module Signing, IMA/EVM, (dm-crypt/LUKS key management)	Digital Signatures, Integrity Verification, Key Wrapping	CONFIG_MODULE_SIG_KEY_TYPE_RSA	Active, crucial for establishing trust
DH	Network Packet Flow (IPsec, TLS Key Exchange)	Key Agreement (Forward Secrecy)	(General Crypto API support)	Shifting towards ephemeral and elliptic curve variants
FFDHE	TLS Key Exchange (Negotiated Finite-Field DH parameters)	Key Agreement (Enhanced Security, Standardization)	CONFIG_CRYPTO_DH_RFC7919_GROUPS	Active, provides standardized DH groups
ECDH	Network Packet Flow (TLS/IPsec Key Exchange), WireGuard VPN	Key Agreement (Efficiency, Forward Secrecy)	CONFIG_CRYPTO_ECDH, ecdh_generic.ko	Active, preferred for performance and security advantages
ECDSA	Kernel Module Signing, Network Packet Flow (TLS/IPsec Digital Signatures)	Digital Signatures, Integrity Verification	CONFIG_MODULE_SIG_KEY_TYPE_ECDSA	Active, multi-algorithm support alongside RSA
EC-RDSA	Kernel Integrity Subsystem (IMA/EVM - Signature Verification)	Digital Signature Verification (Compliance)	CONFIG_CRYPTO_ECRDSA, ecrdsa_generic.ko	Active (verification-only), supports Russian standards
Curve25519	WireGuard VPN (Key Exchange, Mutual Authentication), Crypto API (X25519)	Key Exchange, Authentication (High-performance, Modern)	CONFIG_CRYPTO_CURVE25519_NEON	Active, performance-optimized, core to WireGuard

Table 2: Hash, Digest, and MAC Algorithm Use Cases in the Linux Kernel

Algorithm Name	Type	Primary Kernel Subsystem/Function	Purpose of Use	Current Status/Notes
BLAKE2b	Hash/Digest	IPE (Integrity Policy Enforcement), Random Number Generator	Data Integrity, Random Number Generation	Modern, active, replaces SHA-1
CMAC	MAC	Network Protocols (CIFS, MAC80211, Bluetooth, Kerberos)	Data Integrity, Origin Authentication	Standardized, active
GHASH	Hash/Digest	GCM (Galois/Counter Mode)	Authentication Tag Generation (AES-GCM specific)	Special purpose, performance optimized
HMAC	MAC	IPsec, Kernel Keyring Service, Module Signing, User-space API	Data Integrity, Origin Authentication, Key Protection	Versatile, active
MD4	Hash/Digest	RAID/MD Drivers (Legacy)	Data Structure (Historical/Non-cryptographic)	Deprecated, vulnerable
MD5	Hash/Digest	TCP Sequence Numbers, Syncookies	Data Integrity (Historical/Non-cryptographic)	Deprecated, vulnerable
Michael MIC	MAC	IEEE 802.11i TKIP (WPA)	Message Integrity (Wireless Networking)	Vulnerable, supports legacy standards
RIPEMD-160	Hash/Digest	LUKS (cryptsetup)	Key Derivation (Historical)	Deprecated, replaced by SHA-256
SHA-1	Hash/Digest	Random Number Generator, BPF, Syncookies, Module Signing	Data Integrity, Revision Identification	Vulnerable, being phased out
SHA-224/256	Hash/Digest	IPsec, LUKS, fs-verity, dm-integrity, Module Signing, TPM	Data Integrity, Authentication, Key Sealing	Widely used, active, hardware accelerated
SHA-384/512	Hash/Digest	IPsec, LUKS, fs-verity, dm-integrity, Module Signing, TPM	Data Integrity, Authentication, Key Sealing	Widely used, active, hardware accelerated
SHA-3	Hash/Digest	Module Signing, TPM	Data Integrity, Cryptographic Diversity	Modern, gaining adoption, hardware accelerated
SM3	Hash/Digest	Trusted Cryptographic Keys (TPM)	Key Sealing	National standard integration, active
Streebog	Hash/Digest	(Kernel module existed but obsoleted)	(National standard integration)	National standard, integration status in flux
Whirlpool	Hash/Digest	LUKS (cryptsetup), General Hashing Tools	Data Integrity (Historical)	Deprecated, moved to legacy provider
xxHash	Hash/Digest	BPF, CFI, Hash Tables	High-performance Data Hashing (Non-cryptographic)	Active, performance optimized

Table 3: Block Cipher Algorithm Use Cases in the Linux Kernel

Algorithm	dm-crypt/LUKS	fscrypt	eCryptfs	IPsec/XFRM
AES	D	D	D	D
AES (fixed time)	S	N/A	S	S
Anubis	S	N/A	S	S
ARIA	S	N/A	S	S
Blowfish	S	N/A	S	S
Camellia	S	N/A	S	S
CAST5	S	N/A	S	S
CAST6	S	N/A	S	S
DES / 3DES EDE	S	N/A	S	L
FCrypt	S	N/A	S	S
Khazad	S	N/A	S	S
SEED	S	N/A	S	S
Serpent	S	N/A	S	S
SM4	S	S	S	S
TEA/XTEA/XETA	S	N/A	S	S
Twofish	S	N/A	S	S

Table 4: Hardware Acceleration Support for Cryptographic Algorithms

Algorithm	x86 (AES-NI)	ARMv8 Crypto Ext.	Intel QAT	Other (VIA PadLock, etc.)
AES	Native Support	Native Support	Native Support	Native Support
Serpent	No Support	No Support	No Support	No Support
Twofish	No Support	No Support	No Support	No Support
Camellia	Partial/Opportunistic	No Support	No Support	No Support
ARIA	Partial/Opportunistic	No Support	No Support	No Support
SM4	No Support	Native Support (v8.4+)	Native Support	Native Support (Zhaoxin)
DES / 3DES EDE	No Support	No Support	Native Support	Native Support (Marvell CESA)
Blowfish	No Support	No Support	No Support	No Support
CAST5 / CAST6	No Support	No Support	No Support	No Support
Anubis / Khazad	No Support	No Support	No Support	No Support
SEED	No Support	No Support	No Support	No Support
TEA/XTEA/XETA	No Support	No Support	No Support	No Support

- Signature algorithm
 - ML-DSA (Dilithium) including HashML-DSA and external Mu support
 - ML-DSA (Dilithium) hybrid signature operation with Curve25519 and Curve448
 - SLH-DSA (Sphincs+) including HashSLH-DSA with SHAKE
 - SLH-DSA (Sphincs+) including HashSLH-DSA with Ascon
- Key Encapsulation Mechanism
 - ML-KEM (Kyber) Key Encapsulation Mechanism (KEM)
 - ML-KEM (Kyber) Key Exchange Mechanism (KEX)
 - ML-KEM (Kyber) Integrated Encryption Schema (IES)
 - ML-KEM (Kyber) hybrid KEM / KEX with Curve25519 and Curve448
 - BIKE Key Encapsulation Mechanism (KEM) (deprecated as specified by original authors)
 - HQC Key Encapsulation Mechanism (KEM) (NIST round 4 winner)
- Maintained by Linux Kernel Crypto Maintainer Stephan Mueller:
https://leancrypto.org/leancrypto/cryptographic_algorithms/index.html

- If large robust quantum computers are ever built, they will break RSA and ECC in a few hours.
- Quantum computer attacks will have no practical effect on symmetric crypto (like AES-128).
- NIST has finalized standardization of the most important new quantum-resistant algorithms.
- PQC standardization will continue in NIST, IETF and others in the next five years.
- When to migrate depends for how long protection is required, the value of the target.
- Cloudflare's experience highlights that the transition to PQC is not just a technical challenge — it's a complex process that involves careful consideration of process design, change management, and compatibility across systems.

- NIST, "Constrained Radio Networks, Small Ciphertexts, Signatures, and Non-Interactive Key Exchange" <https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/constrained-radio-networks-pqc2022.pdf>
- New Scientist, "Cryptographers bet cash on when quantum computers will beat encryption" <https://www.newscientist.com/article/2370022-cryptographers-bet-cash-on-when-quantum-computers-will-beat-encryption/>
- NSA, "Announcing the Commercial National Security Algorithm Suite 2.0" https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMMS_PDF
- BSI, ANSSI, Dutch and Swedish NCSA, "Position Paper on Quantum Key Distribution" https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/Quantum_Positionspapier.pdf
- Ericsson Technology Review, "Quantum technology and its impact on security in mobile networks" <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/ensuring-security-in-mobile-networks-post-quantum>
- Ericsson Blog, "Migration to quantum-resistant algorithms in mobile networks" <https://www.ericsson.com/en/blog/2023/2/quantum-resistant-algorithms-mobile-networks>
- NIST, "Post-Quantum Cryptography Project" <https://csrc.nist.gov/projects/post-quantum-cryptography>
- arXiv, "Quantum-Resistant Cryptography" <https://arxiv.org/abs/2112.00399>
- NISTIR 8547, "Transition to Post-Quantum Cryptography Standards" <https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf>
- ANSSI, "Guide des Mécanismes cryptographiques" https://cyber.gouv.fr/sites/default/files/2021/03/anssi-guide-mecanismes_crypto-2.04.pdf
- QUANTUM ATTACKS ON AES: https://ches.iacr.org/2024/Jaques_CHES_2024.pdf

